



Universidad Argentina de la Empresa
Facultad de Ingeniería y Ciencias Exactas

Programación I

Aula 509, Lima 1

Grupo 7

Hotel Boutique

Alumnos

Desarrollador Diego Mora - LU: 1224472

Desarrollador Ramiro Fiol - LU: 1139391

Analista Ignacio Uliczki - LU: 1238960

Jefe Analista Román Oro - LU 1210619

Analista Valentino Zaninetti - LU 1202296

Docentes

Escandell, Gustavo Emmanuel

Selles, Melinda Lujan

Link al repositorio Github: https://github.com/MoraDiego02/Proyecto_Hotel

Presentación del proyecto

Nuestro objetivo principal es desarrollar un sistema de gestión de reservas hoteleras. Todo comienza cuando el usuario ingresa su DNI y selecciona la habitación, piso y ubicación que se va a hospedar. Una vez hecha la elección, el sistema mostrará una matriz donde el usuario podrá ver el piso y habitación a reservar. Luego el usuario deberá abonar el precio de la reserva la cual puede variar dependiendo del día los cuales algunos tienen descuento o recargo además de la forma de pago; si se realiza con tarjeta, tendrá un recargo del 5%. Luego el sistema entregará un comprobante detallado DNI del usuario, piso reservado, habitación y monto final a pagar.

Alcance del proyecto

1. Toma de datos y validación inicial Todo arranca cuando el usuario quiere hacer una reserva. El sistema le va a pedir su DNI, el sistema contiene una lista la cual guarda los DNIS ingresados así no se pueden repetir en un futuro. el tipo de habitación y piso que busca además de la vista las cuales pueden ser al jardín o a la ciudad, también cómo prefiere pagar. Para asegurarnos de que no haya errores de tipeo o datos falsos, el sistema va a validar todos estos ingresos.

2. Asignación de habitaciones Una vez que el cliente está validado, pasamos a la gestión del hotel. Para representar visualmente los pisos y las habitaciones de forma eficiente, vamos a crear una matriz donde se podrá ver el piso y habitación seleccionada por el usuario.

3. Métodos de pago

El precio de la estadía no es igual en todos los casos, sino que el sistema lo calcula en el momento basándose en dos cosas:

- **El día de ingreso:** Si entran en la semana (lunes a jueves), se llevan un 15% de descuento. Si vienen el fin de semana (viernes a domingo), hay un recargo del 10%.
- **El medio de pago:** Si pagan en efectivo el precio queda igual, pero si usan tarjeta de crédito se suma un 5% extra.

Al confirmar la reserva, el sistema le genera al cliente un comprobante claro y detallado. Ahí va a poder ver sus datos, qué habitación y piso le tocó y el precio final.

4. Volver a reservar

En esta instancia el usuario podrá volver a realizar una reserva si así lo desea, si es así el programa se vuelve a ejecutar desde el inicio llamando a la función `main()`. Si no desea otra reserva el sistema se cerrará dejando un mensaje de agradecimiento por la reserva.

Para mantener un orden en el proyecto optamos por un desarrollo completamente modularizado. separamos las tareas en distintas funciones (una para cargar, otra para validar, etc.) logrando un código claro y reutilizable. Además, decidimos que todo el programa va a correr dentro de una función principal llamada `main` ejecutándose con el clásico `if __name__ == "__main__": main()` al final de nuestro archivo Python.

En este apartado está la documentación de cada función del proyecto junto con la explicaciones de uso de:

- Listas avanzadas
- Matrices
- Funciones lambda
- Expresiones regulares

Funciones

Archivo funciones_hotel.py

- `ingresar_dni`: La función `ingresar_dni` solicita al usuario que ingrese su dni y valida que tenga 8 dígitos enteros. si el valor es correcto retorna el DNI ingresado.
- `gestion_reserva`: Esta función gestiona el cálculo del precio final de una reserva. genera un día aleatorio de la semana y aplica un descuento del 20% si este día se encuentra entre el lunes y jueves, o un recargo del 10% si es entre viernes y domingo. Luego solicita al usuario el método de pago (Efectivo o tarjeta) y aplica un recargo adicional del 5% si se selecciona tarjeta. El precio final se calcula usando *reduce* para combinar los multiplicadores y se retorna el monto total a abonar.
- `tipo_habitaciones`: La función muestra por pantalla los tipos de habitaciones que están disponibles. Usa *map* junto con *enumerate* para generar y formatear de forma automática la lista numerada de operaciones.
- `filtrar_ubicaciones_por_habitacion`: Esta función devuelve las ubicaciones disponibles según el tipo de habitación seleccionada. Usa *filter* para seleccionar solo aquellas ubicaciones que permiten la habitación indicada
- `pedir_piso`: Esta función solicita al usuario que ingrese el número de piso para la reserva. Valida que el valor esté entre 1 y 3, y en caso contrario vuelve a pedir el dato. Devuelve el número de piso ingresado
- `pedir_habitacion`: La función solicita al usuario que ingrese el número de habitación que desea. Valida que el valor esté entre 1 y 3, y en caso de error solicita nuevamente el ingreso. Retorna el número de habitación seleccionado.
- `Mostrar_habitaciones`: La función muestra en pantalla el estado de ocupación de las habitaciones organizadas por piso. Usa una matriz donde los valores indican si una habitación está ocupada o disponible, representando las que están ocupadas con una X. Devuelve la matriz.
- `cargar_matriz`: Esta función crea e inicializa una matriz de 3x3 con valores en 0, donde cada fila representa un piso y cada columna una habitación. Retorna la matriz creada.
- `comprobante_reserva`: La función muestra por pantalla un comprobante de la reserva realizada, incluyendo el DNI del cliente, el piso, la habitación seleccionada y el precio final a abonar.
- `inicio`: Esta función muestra un mensaje de bienvenida al usuario al iniciar el sistema

Archivo Hotel_proy.py

- `main`: La función principal del programa. Se encarga de coordinar el flujo general del sistema de reservas, incluyendo dentro la carga de datos del cliente, la selección de piso, habitación y ubicación, la actualización del estado de las habitaciones, el cálculo del precio final y la generación del comprobante de reserva.

Lambda: Se hizo uso de una función lambda para definir una función que genere un número aleatorio entre el 0 y 6. Se asignó esta función a la variable `generar_dia` y luego se invoca como una función común para conseguir un valor que represente un día de la semana.

Matrices: En este proyecto utilizamos las listas bidimensionales mejor conocidas como matrices para representar la ubicación del piso y habitación de la reserva de los usuarios. Esto es visible ya que se marcará con una X la opción elegida previamente.